PROJECT FIELD GUIDE

Employee Management

A beginner-friendly tour of the Java backend, database, React frontend, security, and workflow

START HERE

Follow one feature from Controller to Service to Repository to Entity. You can understand the flow without knowing every Spring annotation first.

Java 21 | Spring Boot 4.1 | MySQL | React 18

Based on the current project files in Downloads

1. WHAT THIS PROJECT DOES

This is a small people operations application. It stores departments, employee records and extra employee details; connects app logins to employees; records attendance; and lets employees request leave for a manager to approve or reject.
The browser is a React single-page app. It sends HTTP requests to a Java Spring Boot REST API. The API applies validation and role checks, runs business rules, and uses Spring Data JPA to read and write MySQL rows.

Browser (React)
| HTTP + JSON, Authorization: Bearer token
v
Spring MVC Controller -> Service -> Repository -> MySQL
|
+--> Email service (leave approval / employee welcome)

WHAT EACH LAYER MEANS IN PLAIN JAVA

Layer | Project examples | Job
Controller | LeaveController | Receives web requests, reads route/body/identity, returns HTTP responses.
Service | LeaveService | Applies rules: date checks, ownership, pending status, DTO conversion.
Repository | LeaveRequestRepository | Database access. Spring Data generates common SQL from method names.
Entity | LeaveRequest | Java object mapped to a database table and its relationships.
DTO | LeaveRequestDTO | Request/response shape sent over HTTP; avoids exposing entity internals.
Good reading habit: for any API URL, locate its Controller mapping first. Then follow the called Service method, repository method, and entity fields.

2. DOMAIN MODEL AND RELATIONSHIPS

Department 1 -------- many Employee 1 -------- 1 EmployeeInfo
|
+-------- many LeaveRequest -------- 1 LeaveType
User 0 or 1 ---------- 0 or 1 Employee

* Department to Employee: one department can contain many employees; each employee has zero or one department.
* Employee to EmployeeInfo: one-to-one. The unique employee_id column prevents two info rows for one employee. New employee creation also creates a blank info row.
* Employee to LeaveRequest: one employee can submit many leave requests. Each request belongs to one employee.
* LeaveRequest to LeaveType: each request chooses one type such as Annual, Sick, or Personal.
* User to Employee: an optional one-to-one login link. A login may have no employee profile until an approver links it.
* Attendance to Employee: each attendance row belongs to one employee and date; the service blocks a second check-in for the same employee on the same day.

WHAT THE JPA ANNOTATIONS DO

@Entity tells JPA that instances map to database rows. @Table chooses a table name. @Id marks the primary key. @GeneratedValue lets MySQL generate IDs. @ManyToOne and @OneToOne describe foreign-key relationships. @JoinColumn names the foreign key.
@Enumerated(EnumType.STRING) stores an enum label such as PENDING instead of a numeric ordinal.
mappedBy marks the inverse side of a relationship. For Employee-EmployeeInfo, the owning side is EmployeeInfo because it holds the employee_id column.

WHY DTOS MATTER

Returning JPA entities directly can expose fields such as a password hash, create huge nested JSON cycles, or run hidden database reads while JSON is being written. The controllers mostly return DTOs such as EmployeeDTO. A DTO is an ordinary Java class used as a clean API contract.

3. ROLES, LOGIN, AND PERMISSIONS

Stored role | Typical ability in current code
ROLE_USER | View own linked employee profile; create and view own leave requests; access own attendance.
ROLE_LEAVE_APPROVER | Manage employees and departments; link logins; view employee data and leave requests; decide leave requests; view attendance across employees.
ROLE_ADMIN | Approver abilities plus account list and role assignment endpoints. Role assignment is protected with @PreAuthorize("hasRole('ADMIN')").
Repository note: ARCHITECTURE.md says there are two roles, but the current Java code also has ROLE_ADMIN. It also describes ddl-auto=update; the current local application.properties has ddl-auto=none. Treat the Java source and current properties as the actual behavior.

LOGIN REQUEST FLOW

* POST /api/auth/login sends username and password.
* Spring's AuthenticationManager checks the submitted password against the stored BCrypt hash.
* On success, JwtUtil returns a signed token; the browser stores it in local storage.
* For later API calls, the React api() helper adds Authorization: Bearer ....
* JwtFilter checks the token, loads the user's role, and sets the authenticated user in Spring Security's request context.
* SecurityConfig and method annotations decide whether that role may call the endpoint.

IMPORTANT IMPLEMENTATION DETAIL

The current JwtUtil creates a fresh signing key when the app starts. This means a backend restart invalidates tokens issued before that restart; the user must sign in again. In a production system, JWT signing material should be a stable secret configured outside the source code.
Public registration sets ROLE_USER in the backend, regardless of a role in the incoming JSON. An administrator can list accounts and change eligible user/approver roles. The frontend's employee editor can link an account to an employee. Employee login and employee record are separate database records linked by a unique relationship.

4. MAIN WORKFLOWS FROM CLICK TO DATABASE

A. CREATE EMPLOYEE AND LINK LOGIN

* Approver opens People and submits employee fields plus optional department and login IDs.
* React sends POST /api/employees with an EmployeeDTO.
* EmployeeController checks role and bean validation.
* EmployeeService finds the requested department/user, rejects an already-linked login, creates an Employee and blank EmployeeInfo, and saves the graph.
* It schedules a welcome email through EmailService.
* The API converts the saved employee to a DTO and returns JSON.

B. EMPLOYEE SUBMITS LEAVE

* React reads available leave types and sends the selected type, dates, and reason.
* LeaveController requires ROLE_USER for POST /api/leaves.
* LeaveService confirms dates are valid, checks the account is linked to an employee, finds the leave type, and saves a PENDING LeaveRequest.
* The employee can view their own request history. An approver sees all requests or filters by employee ID.

C. APPROVER DECIDES LEAVE

* Approver calls PATCH /api/leaves/{id}/decision with APPROVED or REJECTED and optional note.
* The service rejects invalid states and requests already decided.
* It saves the decision and timestamp. On approval, it asks the email service to notify the employee address.

D. ATTENDANCE

Employees use check-in and check-out for their own employee ID. The controller checks that the authenticated user's linked employee ID matches. Approvers/Admin can view by date and may act for employees. The service records today's date and local check-in/out time.

E. START-UP SEEDERS

At application startup, EmployeeInfoSeeder creates missing info rows for older employee rows. LeaveTypeSeeder inserts Annual, Sick, and Personal only when the leave_types table is empty.

5. API MAP

Route | Purpose | Role note
POST /api/auth/register | Create a login. | Public; always ROLE_USER.
POST /api/auth/login | Verify credentials and return JWT. | Public.
GET /api/auth/me | Current login, role, and linked employee ID. | Authenticated.
GET /api/auth/users | List accounts for role management. | Admin only.
PATCH /api/auth/users/{id}/role | Change Employee/Approver role. | Admin only.
GET /api/auth/employee-logins | Accounts available to link to employees. | Approver/Admin.
GET,POST /api/departments | List/create departments. | Approver/Admin.
GET,POST /api/employees | List/create employee records. | Approver/Admin.
GET /api/employees/mine | Current user's employee record. | Employee.
GET,PUT /api/employees/{id} | Read/update employee. | Approver/Admin.
GET,PUT /api/employees/{id}/info | View/edit separate employee details. | Owner can read; Approver/Admin can edit.
GET /api/employees/page|search|department/{id} | Paginated, searched, or department-specific employee list. | Approver/Admin.
GET /api/leaves/types | Available leave categories. | Authenticated.
GET /api/leaves | Own requests or all requests; optional employeeId filter. | Employee sees own; approver sees all.
POST /api/leaves | Submit a pending request. | Employee.
PATCH /api/leaves/{id}/decision | Approve/reject a pending request. | Approver/Admin.
/api/attendance/... | Check-in, check-out, own history, date reports and ranges. | Owner or Approver/Admin according to route.
GET /actuator/health, /actuator/startup | Health and startup diagnostics. | Protected by authenticated default rule.

6. JAVA FILE GUIDE - APPLICATION AND CONFIGURATION

File | What it is for | Java idea to notice
EmployeeManagementApplication.java | Main entry point. Boots Spring; turns on async email and records startup steps. | main(), annotations, app bootstrap.
config/SecurityConfig.java | JWT filter chain, CORS, stateless sessions, URL rules, BCrypt bean, AuthenticationManager. | Configuration class, beans, fluent API, lambdas.
config/EmployeeInfoSeeder.java | On startup, add missing EmployeeInfo rows for old employees. | CommandLineRunner lambda, repository iteration.
config/LeaveTypeSeeder.java | Seed baseline leave types when table is empty. | Conditional initialization.

SECURITY TERMS

* @Configuration: class contributes objects to Spring's application context.
* @Bean: method result becomes a managed object.
* @RestController: controller methods return data directly as HTTP response bodies.
* @PreAuthorize: method-level permission expression; hasRole('ADMIN') checks the authority **ROLE_ADMIN.**
* SessionCreationPolicy.STATELESS: server does not keep a logged-in session; each request supplies a token.
* CORS controls which browser origins can call the API. It is not the same thing as authentication.

7. JAVA FILE GUIDE - CONTROLLERS AND SERVICES

File | Responsibility
controller/AuthController.java | Registration, login, current user, account list, role changes, logins for employee linking.
controller/DepartmentController.java | Department create/list endpoints; approver/admin gate.
controller/EmployeeController.java | Employee CRUD, current employee, pages/search/department endpoints; DTO validation.
controller/EmployeeInfoController.java | Separate employee-info read and approver/admin update routes.
controller/LeaveController.java | Leave types, request listing/creation, decisions.
controller/AttendanceController.java | Check-in/out and own/admin attendance queries; verifies employee ownership.
service/EmployeeService.java | Maps EmployeeDTO to entity and back; manages department/user links; creates info; page/search filters.
service/EmployeeInfoService.java | Checks who may view profile details; creates missing info; saves fields and maps DTO.
service/LeaveService.java | Leave validation, ownership, request creation, approvals, history, email trigger.
service/AttendanceService.java | Attendance business logic and DTO conversion.
service/EmailService.java | Asynchronous welcome/approval email with SMTP configuration and error logging.

Controllers should stay thin: they translate HTTP into Java method calls. Services are the best place to look for business rules. The app uses both constructor injection and @Autowired field injection; constructor injection makes required dependencies explicit and is usually easier to unit-test.

@Transactional groups database operations into one transaction. A read-only transaction marks query operations as read-only. If a runtime exception occurs in a transaction, Spring generally rolls back the work.

8. JAVA FILE GUIDE - ENTITIES, DTOS, REPOSITORIES

ENTITIES

Entity file | Table / main fields
entity/User.java | users: username, password hash, role, optional employee link.
entity/Department.java | departments: unique name and employees collection.
entity/Employee.java | employees: name, email, salary, department, login, one info record.
entity/EmployeeInfo.java | employee_info: job title, phone, address, birth/hire dates, emergency contact.
entity/LeaveRequest.java | leave_requests: employee, type, date range, reason, status and decision data.
entity/LeaveType.java | leave_types: unique name and description.
entity/Attendance.java | attendance: employee, date, check-in/out and status.
entity/LeaveStatus.java | Allowed leave states: PENDING, APPROVED, REJECTED.
entity/AttendanceStatus.java | Attendance labels: PRESENT, ABSENT, HALF_DAY, ON_LEAVE.
@Data, @NoArgsConstructor, and @AllArgsConstructor come from Lombok and generate boilerplate such as getters, setters, and constructors during compilation.

DTO FILES

DTO | Use
dto/EmployeeDTO.java | Employee fields plus departmentId/name and userId; validates names, email and positive salary.
dto/EmployeeInfoDTO.java | Profile details with maximum string lengths.
dto/LeaveRequestDTO.java | Leave request payload/response and validation for type, dates, reason.
dto/LeaveDecisionDTO.java | Decision status and optional note.
dto/LeaveTypeDTO.java | Leave category returned to UI.
dto/AttendanceDTO.java | Attendance response including employee name.

REPOSITORIES

Each repository extends JpaRepository<Entity, Long>, which provides save/find/delete/page methods. Additional methods are derived from names: findByEmployeeIdAndDate means filter by employee's ID and date. @Query is used for a custom JPQL query in EmployeeRepository. @EntityGraph asks JPA to load the employee link together with user rows.
repository/UserRepository.java | Username lookups, role count, login summaries with linked employee.
repository/EmployeeRepository.java | Employee pages, first-name search, department query.
repository/DepartmentRepository.java | Department CRUD.
repository/EmployeeInfoRepository.java | Info lookup by employee ID.
repository/LeaveRequestRepository.java | Requests by employee/status and recent-first lists.
repository/LeaveTypeRepository.java | Leave type CRUD.
repository/AttendanceRepository.java | Attendance lookup by employee/date and date range.

9. JAVA FILE GUIDE - SECURITY AND ERRORS

File | Purpose
security/JwtUtil.java | Signs, validates, and reads the username from JWTs; current token lifetime is 10 hours.
security/JwtFilter.java | Runs once per request; reads Bearer token; installs Authentication into SecurityContext.
security/UserDetailsServiceImpl.java | Loads a user from the database and converts the stored role into a Spring authority.
exception/ResourceNotFoundException.java | Simple application exception for missing records.
exception/GlobalExceptionHandler.java | Maps not-found, validation, access, conflict and generic exceptions into JSON HTTP responses.

HOW VALIDATION BECOMES AN ERROR RESPONSE

A controller argument marked @Valid causes Jakarta Validation annotations on the DTO to run before the service method. If validation fails, Spring throws a validation exception. The global handler formats field messages and sends HTTP 400 Bad Request.

USEFUL HTTP STATUS MEANINGS

Status | Meaning in this app
200 | Request succeeded (read/update/login depending on endpoint).
201 | Created employee or leave request.
400 | Invalid fields or business input such as dates.
401 | Missing/invalid login authentication or expired token.
403 | Authenticated identity lacks the role or ownership permission.
404 | Requested entity or route does not exist.
409 | Conflict, such as already checked in or duplicate linked account.
500 | Unexpected server error; check Render logs for the exception stack trace.

10. FRONTEND, DATABASE, AND DEPLOYMENT FILES

File | Purpose
frontend/src/App.jsx | Main UI: login/register, People, departments, profile details, leave and attendance views; API helper stores/sends JWT.
frontend/src/main.jsx | Creates the React root and renders App with styles.
frontend/src/styles.css | Layout, typography, responsive behavior, light/dark themes.
frontend/vite.config.js | Vite development server and local /api proxy to port 8080.
frontend/package.json / package-lock.json | Frontend dependencies and repeatable npm dependency versions.
frontend/index.html | HTML shell that holds the React root element.
frontend/Dockerfile | Builds static frontend assets, then serves them with Nginx.
frontend/nginx.conf | Serves the SPA and proxies /api/ to backend container.
pom.xml | Maven build, Java 21, Spring Boot 4.1 and dependencies (Web MVC, JPA, Security, validation, MySQL, JWT, Actuator, mail, OpenAPI).
mvnw / .mvn/ | Maven Wrapper so contributors can invoke the Maven version without installing it globally.
src/main/resources/application.properties | Database, JPA, port, CORS, API docs, actuator, mail, Hikari pool settings.
Dockerfile | Multi-stage Maven build then small Java 21 runtime image.
compose.yaml | Defines database, backend and frontend containers plus ports and dependencies.
.env.example | Template names for local database/mail settings. Copy to ignored .env; never commit real secrets.
.github/workflows/docker-publish.yml | On push to prod, builds and publishes backend/frontend container images to GHCR; it does not itself deploy Render.
ARCHITECTURE.md / DOCKER.md | Human-maintained architecture and local Compose instructions; check architecture doc against current code when roles/config change.
src/test/java/.../*Test.java | Existing Spring/controller and service tests; this guide creation did not run them.

Compose configuration to verify: the current compose.yaml starts a local MySQL container but sets the backend JDBC URL to the Aiven host. That means the backend is configured for Aiven, despite also waiting for the local db service. Confirm this is intentional before using Compose as a local-only stack.

11. CURRENT DATABASE SETTINGS AND WORKFLOW

Current working-tree settings are MySQL-oriented: the JDBC driver is MySQL Connector/J, the dialect is MySQLDialect, and the server uses ${PORT:8080} so a host-provided PORT can be honored.

* spring.main.lazy-initialization=true: defers creation of many beans until needed. This can shorten reported startup but may move work to the first request.
* spring.jpa.hibernate.ddl-auto=none: Hibernate will not create/update tables. Database tables and schema must already match Java entities.
* spring.jpa.open-in-view=false: prevents lazy database reads from happening while a web response is being rendered.
* hibernate.boot.allow_jdbc_metadata_access=false: disables Hibernate's metadata query during boot; explicit database dialect is therefore configured.

Because ddl-auto is none, adding an entity field/table needs a deliberate MySQL schema migration before deploy. The previous automatic update behavior is not active in this local working tree.

RUN LOCALLY (TYPICAL)

* Install JDK 21, Node/npm, and MySQL or configure a database URL.
* Set environment values for datasource URL/user/password and optional SMTP.
* Start backend from project root with ./mvnw spring-boot:run.
* Start frontend with cd frontend && npm install && npm run dev.
* Open Vite's local URL, normally http://localhost:5173. Vite proxies /api to localhost:8080.
* Register a normal account. It starts with ROLE_USER. An administrator must grant approver role through the protected admin flow/database setup. An approver creates an employee and links the login from People.

DEBUG REQUEST SEQUENCE

* Browser DevTools Network: check request URL, method, status, request JSON and response body.
* Check backend logs at that timestamp. SQL output is visible because spring.jpa.show-sql=true.
* For 401, log in again and confirm bearer token header. This project currently regenerates JWT key on process restart.
* For 403, token may be accepted but role/ownership/Actuator access denies the action.
* For a slow first call, compare Render cold-start logs to app-ready time. The startup endpoint recorded ~2m53s in the sample shared in this conversation. That measures app startup, not the request duration after startup.

12. INTERVIEW QUESTIONS WITH EXPLANATIONS (1-10)

1. Explain this project's architecture.
It is a layered Spring Boot REST application: controllers handle HTTP, services contain business rules, repositories access persistence, entities map relational data, DTOs define API data, and a React client consumes the API. This separation keeps web concerns out of business logic and SQL concerns out of controllers.
2. What is dependency injection?
Instead of a class constructing its own database repository, Spring creates and supplies it. This reduces coupling and makes implementations easier to replace or mock. In this project, services receive repositories through constructors or @Autowired.
3. What is inversion of control?
Normally application code decides when to create objects. With Spring, the framework owns object creation and lifecycle, then calls your code. Dependency injection is a practical form of this idea.
4. What is a REST controller?
A class marked @RestController maps HTTP requests to Java methods and serializes returned objects as JSON. @GetMapping, @PostMapping, and similar annotations connect routes and verbs.
5. Why use a service layer?
It gives business rules a home independent of HTTP. For example, LeaveService validates dates and prevents a second decision; those rules can be called from different controllers or tested without starting a browser.
6. What is a repository?
A repository abstracts persistence. Extending JpaRepository gives standard CRUD and paging operations. Spring Data also creates queries from method names, such as findByEmployeeIdAndDate.
7. What is JPA and what is Hibernate?
JPA is the Java standard/API for object-relational mapping. Hibernate is the implementation used here. An entity is a Java object that Hibernate maps to rows, columns and foreign keys.
8. Why are DTOs used?
DTOs define the JSON contract and prevent exposing persistence details. They can flatten relationships into IDs/names, validate incoming values, and avoid cycles or accidental sensitive fields in entity serialization.
9. Explain one-to-many versus many-to-one.
One department can have many employees, so Department declares a collection with @OneToMany; each employee points to one department with @ManyToOne. The database foreign key lives on the employee row.
10. What does @Transactional do?
It asks Spring to run a method inside a database transaction. Related changes commit together on success and roll back together on qualifying failures. The employee creation flow can save employee and info consistently.

13. INTERVIEW QUESTIONS WITH EXPLANATIONS (11-20)

11. What is JWT authentication?
The server verifies username/password once and returns a signed token with identity claims. The client sends it on later requests. The server validates the signature and expiry instead of keeping a session map. This app's token currently has a 10-hour lifetime.
12. Authentication versus authorization?
Authentication answers "who are you?"; authorization answers "may you do this?". Login and JWT validation authenticate. Role annotations and ownership checks authorize.
13. What does stateless mean in Spring Security?
The server does not store an HTTP login session between requests. Each protected API request carries credentials, here a Bearer JWT.
14. What is BCrypt and why hash passwords?
BCrypt is a deliberately slow password hashing function with a salt. The server stores the hash, not the raw password, and verifies a login by comparing the candidate to the hash. Passwords should never be logged or stored in plain text.
15. What is CORS?
Browser policy controlling which origins may call a server from JavaScript. It is separate from API authorization: a CORS-enabled request can still get 401/403. Preflight is commonly an OPTIONS request.
16. What do @Valid and validation annotations do?
@Valid tells Spring to validate a request DTO. Annotations like @NotBlank, @Email, and @Positive express input constraints before business logic runs.
17. Why enum values with EnumType.STRING?
It stores readable labels such as APPROVED. Ordinal storage uses numbers and can silently change meaning when enum order changes.
18. What does lazy fetching mean?
A related entity is fetched only when needed; eager fetching retrieves it immediately. Lazy fetching can reduce unnecessary SQL but requires an open persistence context or an explicit fetch plan when data is later accessed.
19. What is the N+1 query problem?
One query loads a list and then one additional query per row loads a relationship, creating N+1 total queries. It can be solved with fetch joins, entity graphs, batch fetching or DTO queries. Inspect SQL logs when lists are slow.
20. What is a database migration?
A versioned, deliberate change to a schema, such as adding a column or table. With current ddl-auto=none, Hibernate will not apply schema changes automatically, so a migration or manual SQL must precede the code that depends on it.

14. INTERVIEW QUESTIONS WITH EXPLANATIONS (21-30)

21. Why use @RestControllerAdvice?
It centralizes exception-to-HTTP-response mapping. Instead of repeating try/catch in every controller, the global handler can make 404, 400 and other error JSON consistent.
22. What is the difference between 401 and 403?
401 means the request is unauthenticated or credentials are invalid. 403 means the identity is known but access is denied. In practice, inspect the actual security configuration and response body because custom handlers can affect the exact response.
23. What does @PreAuthorize do?
It checks a SpEL expression before a method runs. hasRole('ADMIN') expects authority ROLE_ADMIN by convention. It is useful for method-specific controls in addition to URL rules.
24. Why separate Employee and EmployeeInfo?
The main employee record contains work identity and organization links; EmployeeInfo contains additional personal/job details. A one-to-one relation lets those concerns and their API page stay separate, while the unique foreign key enforces one info row per employee.
25. How is employee login linking protected?
The account ID is supplied by the approver's UI. EmployeeService checks the account exists and is not already linked to another employee before saving. The unique one-to-one key gives the database a second line of defense.
26. What does @Async do here?
It runs the annotated email method on a Spring-managed executor rather than blocking the caller until SMTP finishes. Async work needs @EnableAsync. Failures should be logged because they no longer return as a normal HTTP error.
27. What is a seeder/CommandLineRunner?
A startup callback. It is used here to provide default leave types and repair missing info records for legacy employees. Seeders should be idempotent so each restart does not duplicate rows.
28. How does the React app call the API?
The shared api() function builds URLs, adds JSON content type and Bearer token, parses errors, and returns JSON. In development Vite proxies /api to Spring Boot; Nginx does the same in the container setup.
29. What does Docker multi-stage build mean?
The backend image first uses Maven/JDK to compile and package, then copies only the jar into a smaller Java runtime image. The frontend builds with Node, then copies static files into Nginx. Build tools are omitted from runtime images.
30. How would you debug a slow registration?
Separate cold start from request execution. Compare request timestamps to Render's application-ready log. If the service is already ready, add timing around password hashing, repository reads/writes and external calls, and compare SQL query timings. The startup timeline measures initialization, not endpoint latency.

15. A PRACTICAL STUDY PATH

* Read EmployeeManagementApplication.java and identify the framework entry point.
* Trace POST /api/leaves across LeaveController, LeaveService, LeaveRequestRepository, LeaveRequest, and LeaveRequestDTO.
* Read SecurityConfig, then JwtFilter and JwtUtil to follow a protected request.
* Trace employee creation through the frontend payload and EmployeeService; observe how IDs connect departments and user logins.
* Read application.properties and explain which values come from environment variables.
* Sketch the database relationships and explain which entity owns each foreign key.
* Practice answering interview questions 1, 2, 5, 8, 11, 12, 15, 16, 20 and 30 in your own words.

JAVA CONCEPTS TO REFRESH AS YOU READ

Classes and objects; interfaces; constructors; generics such as List<Employee>; exceptions; lambdas; streams; annotations; enums; access modifiers; and dependency injection. Spring annotations do not replace Java - they give framework instructions about your Java classes.

USEFUL FILE STARTING POINTS

* Business flow: service/LeaveService.java
* Permissions: config/SecurityConfig.java and controller @PreAuthorize annotations
* Database model: entity/
* HTTP contract: controller/ plus dto/
* Run/deploy: application.properties, Dockerfile, compose.yaml, and GitHub workflow

This guide describes the repository's current local working tree at creation time. Deployment branches and Render configuration can differ. Do not treat secrets from local environment files as part of the guide.